

Resource management

Luca Parisi, EPCC

l.parisi@epcc.ed.ac.uk

Resources

In a program you often need to manage things like:

- memory
- open files
- GPUs
- network sockets

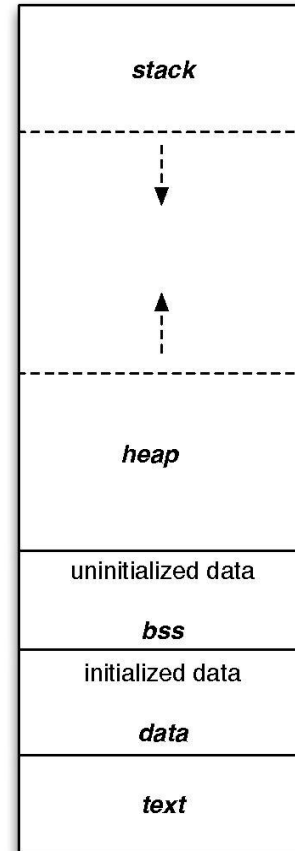
Memory: overview

Modern OSes give each process a flat address space

Each byte can be accessed by its address, which is just a number.

For most purposes this can be considered in two parts:

- stack
- heap



Memory: stack

In C++ (and C and many other compiled languages) local variables are stored in the **stack**.

As your program runs, the local variables that are defined, get space allocated by the compiler relative to the current stack frame.

Each time you call a function, you start a new stack frame by incrementing the stack pointer.

Variables are implemented as offsets from this, so allocating a local variable has no run-time cost.

When you return from a function, the stack pointer is updated and deallocation also has no cost

These allocations are called *static* because they have to be prepared at compile time

Memory: heap

The language and OS also make available some memory for dynamic allocation: the *heap*

You need to request some memory to store an object and then give it back when you are finished

In C++ this uses the `new` and `delete` keywords

```
int main()
{
    // This memory for 3 integers
    // is allocated on heap.
    int *ptr = new int[3]{1,2,3};

    // To keep things tidy we need to manually call delete
    delete[] ptr;
}
```

Pointers in C++

In C++ you can get a pointer to an object using & (the *address of* operator)

You can read or write the value-that-is-pointed-to with * (the *dereference* operator)

```
int main() {  
    int i = 99;  
    int* i_ptr = &i;  
  
    *i_ptr += 1;  
    std::cout << i << std::endl;  
}
```

Pointers vs references

Pointers are a lot like references, but they are not guaranteed to be initialised to a valid value!

It is valid to assign an arbitrary value to a pointer, but actually using it is **undefined behaviour** - typically a crash but could be silent corruption of data.

```
int main() {  
    int* i_ptr = 0xdeadbeef;  
  
    *i_ptr += 1;  
    std::cout << i << std::endl;  
}
```

Use pointers with great caution!

Pointers vs references

Pointers are a lot like references, but they are not guaranteed to be initialised to a valid value!

It is valid to assign an arbitrary value to a pointer, but actually using it is **undefined behaviour** - typically a crash but could be silent corruption of data.

```
int* make_bad_pointer() {  
    int i = 42;  
    return &i;  
}
```

Returning a pointer to a local variable is a recipe for problems

Pointers to dynamically allocated memory

In C++ you can manually create instances of objects dynamically with `new` and try to remember to destroy them with `delete`

```
int main()
{
    // Memory for an integer is allocated on heap.
    int *ptr = new int{};
    *ptr = 7; // Put a value in that memory location

    // Initialize a dynamic array
    int* array_ptr = new int[5]{ 9, 7, 5, 3, 1 };

    // To keep things tidy we need to manually call delete
    delete ptr;
    delete[] array_ptr;
}
```

But doing so is a recipe for problems!

Pointers exercise

In your clone of this repository, find the 4-pointers exercise and list the files

```
$ cd archer2-cpp/exercises/4-pointers
$ ls
Makefile    pointers.cpp
```

Follow the instructions in `pointers.cpp` to test your understanding of pointers.

After each step, you should print variables as directed and check the results are what you expect.

Reminder:

- You can get a pointer to an object using `&` (the *address of* operator)
- You can read or write the value-that-is-pointed-to with `*` (the *dereference* operator)